# NOC2AXI

DESIGN SPEC

RICHARD ROZE

# Table of Contents


1 Introduction ... 3
1.1 Feature Summary ... 3
1.2 Changes from Previous Version ... 3
1.2.1 NOC Interfaces ... 3
1.2.2 AXI Interface ... 3
1.2.3 Data Path Width ... 3
1.2.4 CDC Modules ... 3
2 General Description ... 4
2.1 Block Diagram ... 4
2.2 High Level Data Flow ... 4
2.2.1 NOC to AXI ... 4
2.2.2 AXI to NOC ... 4
2.2.3 APB Traffic ... 4
2.3 AXI Interoperability ... 5
2.3.1 General ... 5
2.3.2 AXI Slave (AXI-to-NOC) Interface ... 5
2.4 NOC Router ... 6
2.5 NIU ... 6
2.5.1 NOC-to-AXI Slave Write ... 7
2.5.2 NOC-to-AXI Slave Read ... 7
2.5.3 AXI-to-NOC Master Write ... 7
2.5.4 AXI-to-NOC Master Read ... 8
2.6 Address Translation ... 9
2.6.1 Without Address Translation (ADDR_TRANSLATION_ON=0 ... 9
2.6.2 With Address Translation (ADDR_TRANSLATION_ON=1) ... 9
2.7 AXI CDC ... 10
2.8 Register Infrastructure ... 10
2.9 Latency ... 10
3 Signal Description ... 11
4 Software Interface ... 13
4.1 Address Map ... 13
4.2 Programming Sequences ... 13
5 Implementation ... 14
5.1 Top-Level Parameters ... 14
5.2 Clocks ... 14
5.2.1 Clock Gating ... 15
5.3 Resets ... 15
5.4 Speed and Size ... 16
5.4.1 Area ... 16
5.5 Memories ... 16

# 1 Introduction

This document is the design spec for the NOC2AXI subsystem. The NOC2AXI subsystem implements an interface between the standard Tenstorrent NOC fabric and an AXI4 interface.

## 1.1 Feature Summary

- 2048b NOC interface
- 512b or 1024b AXI4 interface

## 1.2 Changes from Previous Version

This document describes a version of the NOC2AXI  that is being modified from its implementation in Black Hole for use in Quasar. This section lists the main set of changes in this version, not including any changes that are not visible externally.

### 1.2.1 NOC Interfaces

#### *1.2.1.1 Number of Interfaces*

The Black Hole version of NOC2AXI supported two input and two output NOC interfaces. The Quasar version of NOC2AXI supports four input and four output NOC interfaces.

#### *1.2.1.2 Data Path Width*

The Black Hole version of NOC2AXI had a data width of 512 bits for its NOC interfaces. The Quasar version of NOC2AXI uses a data width of 2048 bits.

In Black Hole the NOC data path width was identical to the data path width of the AXI interface. Support was added in Quasar to support a NOC data path width that is different from that of the AXI interface.

### 1.2.2 AXI Interface

#### *1.2.2.1 Native AXI4 Support*

The Black Hole version of NOC2AXI had limited support for AXI4. This version improves the buffering to support maximum data rate for maximum transaction burst sizes.

### 1.2.3 Data Path Width

The Black Hole version of NOC2AXI had a data width of 512 bits for its AXI interfaces. The Quasar version of NOC2AXI supports an AXI data path width of either 512 bits of 1024 bits.

### 1.2.4 CDC Modules

The Black Hole design used DesignWare IP to perform clock domain crossing (CDC) between the NOC and AXI clock domains. This version replaces the DesignWare IP with Tenstorrent designs.

# 2 General Description

This section describes the functionality and structure of the NOC2AXI SS.

## 2.1 Block Diagram

Figure 1 shows an overview of the functions implemented by the NOC2AXI SS.

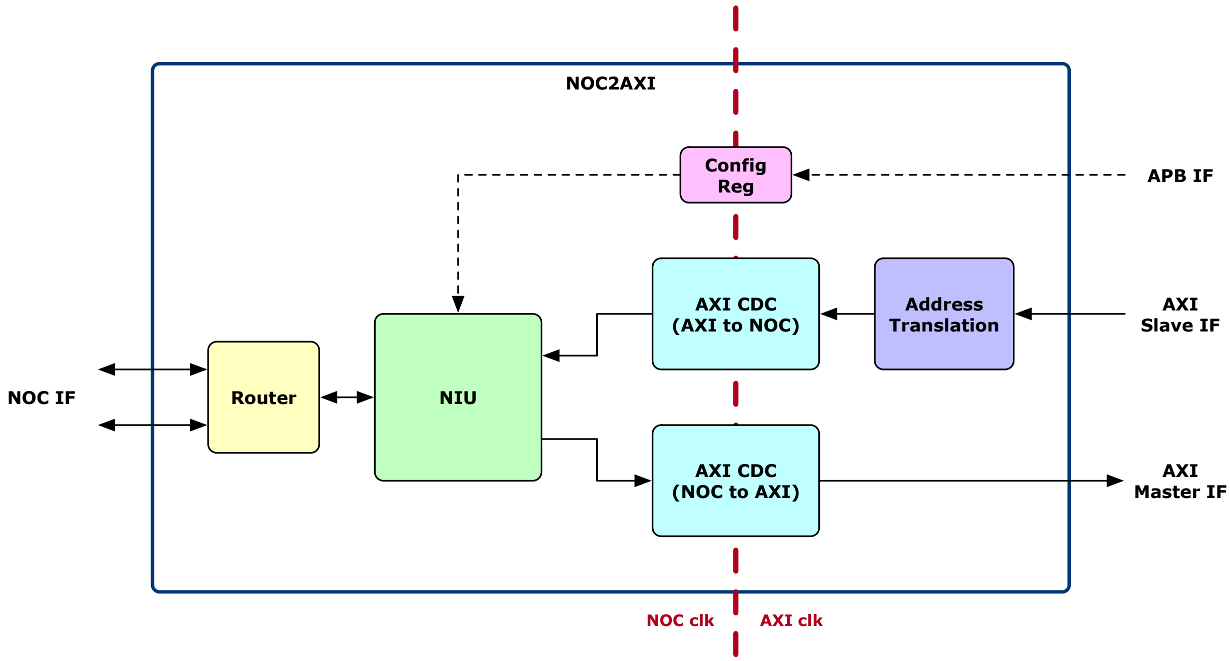


**Figure 1: NOC2AXI SS Functions**

## 2.2 High Level Data Flow

### 2.2.1 NOC to AXI

Transactions originating in the NOC grid are converted and issued on the AXI master interface.

Since AXI transactions are limited to 4 KB address boundaries, a single NOC transaction will be split into multiple transactions on the AXI interface if the address limit is crossed.

#### *2.2.1.1 Limitations*

All transactions originating in the NOC must use both target and return addresses that are aligned to the width of the AXI data bus.

### 2.2.2 AXI to NOC

Transactions originating from an external AXI master are converted and issued on the NOC master interface.

### 2.2.3 APB Traffic

APB transactions are received and can target either a small set of NOC2AXI local registers or the NIU register space.

## 2.3 AXI Interoperability

This section defines the capabilities and limitations of the AXI interfaces.

### 2.3.1 General

#### *2.3.1.1 Protocol Version*

The NOC2AXI SS supports the AMBA AXI4 protocol as defined in Issue E of the *AMBA AXI and ACE Protocol Specification*.

#### *2.3.1.2 Bit Widths*

Table 1 shows the width of certain AXI interface signals.

| Signal | Bit Width |
|---|---|
| AWADDR, ARADDR | 88 |
| AWID, ARID | 12 |
| AWUSER, ARUSER | 38 |

**Table 1: Signal Bit Widths**

#### *2.3.1.3 Feature Support*

##### 2.3.1.3.1 Multiple Region Signaling

The NOC2AXI SS does not support multiple region signaling. The AxREGION signals are always driven to zero by the master interface and are ignored by the slave interface.

##### 2.3.1.3.2 Locked Accesses

The NOC2AXI SS does not support locked accesses. The AxLOCK signals are always driven to zero by the master interface and are ignored by the slave interface.

##### 2.3.1.3.3 Memory Types

The NOC2AXI SS does not support any memory type other than Device Non-bufferable. The AxCACHE signals are always driven to zero by the master interface and are ignored by the slave interface.

##### 2.3.1.3.4 Access Permissions

The NOC2AXI SS does not support non-default access permissions. The AxPROT signals are always driven to zero by the master interface and are ignored by the slave interface.

##### 2.3.1.3.5 QoS Signaling

The NOC2AXI SS does not support QoS signaling. The AxQOS signals are always driven to zero by the master interface and are ignored by the slave interface.

### 2.3.2 AXI Slave (AXI-to-NOC) Interface

The AXI slave interface is classified as a Peripheral Slave.

##### 2.3.2.1.1 User-Defined Signaling

If the address translation capability of the NOC2AXI is not enabled, the NOC2AXI SS makes use of the user-defined signals on each of the AXI slave interface address channels.  Table 1 shows the usage of  the bits in the AWUSER signal. Some signals are used only by AWUSER or ARUSER, but not both.

| Bit | Field Description | AWUSER field read by Slave? | ARUSER field read by Slave? |
|---|---|---|---|
| AxUSER[0] | bcast | Y | N |
| AXUSER[1] | strict order | Y | Y |
| AxUSER[2] | posted | Y | N |
| AxUSER[3] | linked | Y | Y |
| AxUSER[4] | static | Y | Y |
| AxUSER[5] | [unused] | N | N |
| AxUSER[8:6] | static vc | Y | Y |
| AxUSER[16:9] | strided | Y | N |
| AxUSER[21:17] | exclude coord x | Y | N |
| AxUSER[25:22] | exclude coord y | Y | N |
| AxUSER[26] | exclude dir x | Y | N |
| AxUSER[27] | exclude dir y | Y | N |
| AxUSER[28] | exclude enable | Y | N |
| AxUSER[29] | exclude routing option | Y | N |
| AWUSER[37:30] | num dest | Y | N |

**Table 2: AXI Slave User-Defined Signaling Bits**

The user-defined signals are not implemented for the AXI master interface.

#### *2.3.2.2 Limitations*

The slave interface does not support arbitrary WSTRB values during multi-transfer burst transactions. For write transactions where the burst length is greater than one, the bytes written must form a contiguous sequence from the lowest-address byte to the highest-address byte; there cannot be any "holes" in the sequence of bytes written. For transactions where the burst length is one, any pattern of bytes may be written.

## 2.4 NOC Router

Identical to the design in the Tensix core. See the NOC design spec for more information on the NOC router.

## 2.5 NIU

The NIU acts as the interface between the NOC router and the AXI interface (and internal register space). Figure 2 shows the main subblocks of the NIU.

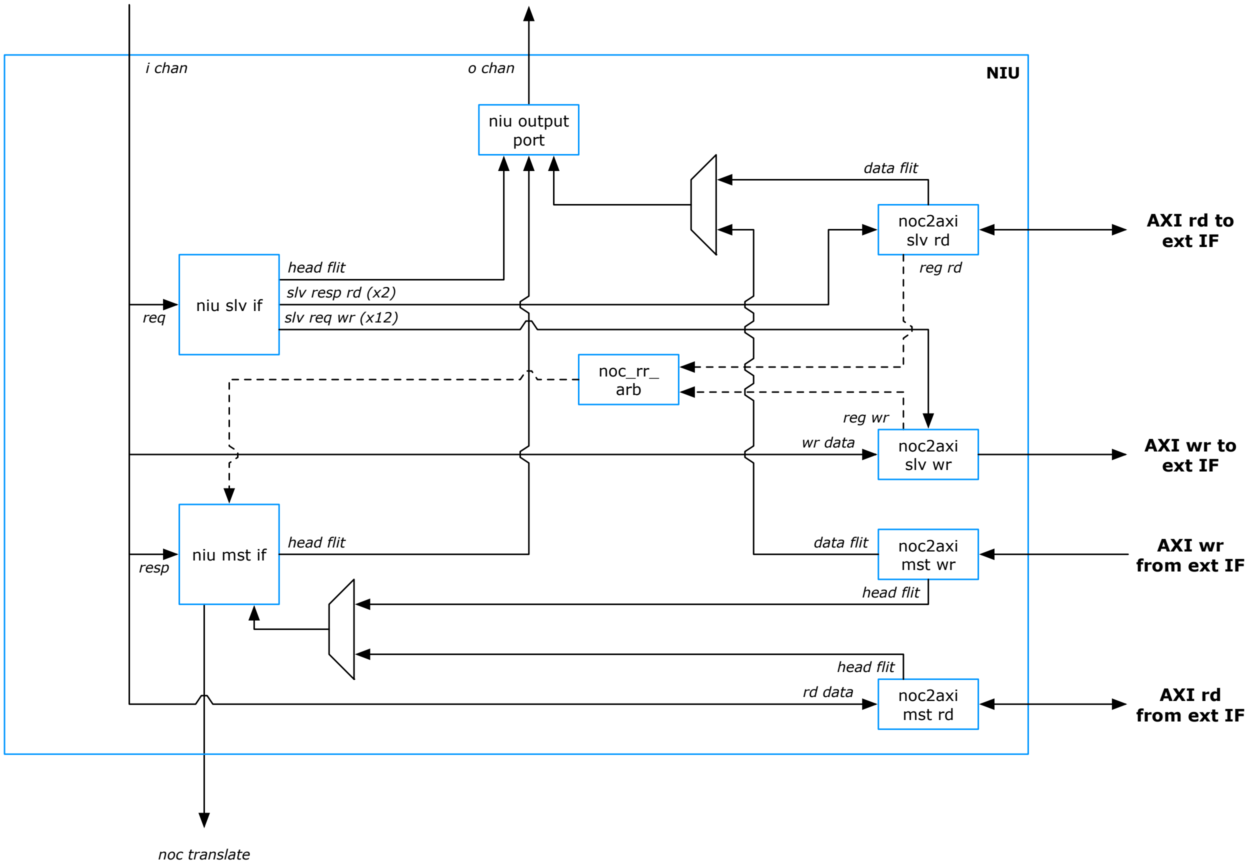


Figure 2: NIU

### 2.5.1 NOC-to-AXI Slave Write

The NOC-to-AXI Slave Write module ("N2ASW") receives write transactions from the NOC and sends corresponding write transactions to the AXI Master interface.

Write transactions issued on the AXI interface use one of twelve different values for the address write ID (AWID).

### 2.5.2 NOC-to-AXI Slave Read

The NOC-to-AXI Slave Read module ("N2ASR") receives read requests from the NOC and sends corresponding read transactions to the AXI Master interface.

All read transactions issued on the AXI interface use the same address read ID (ARID) value of zero. Therefore all read data must be returned in the order in which the read requests are issued.

### 2.5.3 AXI-to-NOC Master Write

The AXI-to-NOC Master Write module ("A2NMW") receives incoming AXI write transactions from an external AXI master and sends corresponding NOC transactions to the NIU Master Interface. The A2NMW can support up to four outstanding write transactions.

The processing of read transactions by the A2NMW is fairly basic.

**Wait for all transaction data:** Write data will not begin to be transferred on the NOC interface until all of the data for that transaction has been received on the AXI interface and is present in the A2NMW's write buffer. This is required due to the fact that the header flit of the NOC transaction requires an exact byte count, and this information will not be known until the last beat of AXI write data is received.

**No out-of-order processing:** The A2NMW processes write transactions in the order in which they are received, including for write responses on the AXI interface. Write response data cannot be returned out of order, regardless of the AXI ID values. This means that even if a transaction that was issued later completes early on the NOC interface, its response cannot be returned on the AXI interface until all previously-issued AXI write requests have their write responses returned on the AXI interface.

#### *2.5.3.1 Write Buffer*

A memory buffer of size 64 words x 2048 bits (total 16 KB storage) stores the write data incoming from the AXI interface. This size is chosen to support four AXI4 transactions, where each transaction is a maximum of 4 KB in size, per the AXI specification. Each transaction must finish being written to the write buffer before it can start being read out, due to the fact that the total amount of data is not known until the end of the AXI transaction.

The width of this write buffer matches the width of the NOC data path, which is wider than the AXI data path. Incoming AXI data beats are consolidated in this buffer to fill the 256-byte flit size.

### 2.5.4 AXI-to-NOC Master Read

The AXI-to-NOC Master Read module ("A2NMR") receives incoming AXI read transactions from an external AXI master and sends corresponding NOC transactions to the NIU Master Interface. The A2NMR can support up to 16 outstanding read transactions.

The processing of read transactions by the A2NMR is fairly basic, in particular with respect to how and when read data is returned on the AXI read response channel.

**Wait for all data:** Read data will not begin to be returned on the AXI interface until all of the data has been provided on the NOC interface and is present in the A2NMR's read buffer. This means that for long bursts, even if the first beats of data are available early, the start of the read response transaction will be delayed until the final beat of data is available.

**No out-of-order processing:** The A2NMR processes read transactions in the order in which they are received, including for read responses on the AXI interface. Read response data cannot be returned out of order. This means that even if a transaction that was issued later completes early on the NOC interface, its data cannot be returned on the AXI interface until all previously-issued AXI read requests have their data returned on the AXI interface.

#### *2.5.4.1 Read Buffer*

A memory buffer of size 256 words x 2048 bits (total 64 KB storage) stores the read data coming from the NOC. This size is chosen to support sixteen AXI4 transactions, where each transaction is a maximum of 4 KB in size, per the AXI specification.

The width of this read buffer matches the width of the NOC data path, which is wider than the AXI data path.

## 2.6 Address Translation

The NOC2AXI SS supports translation of the incoming AXI addresses on its AXI slave interface. The decision as to whether the address translation functionality is available is made via a top-level SystemVerilog parameter named `ADDR_TRANSLATION_ON`.

### 2.6.1 Without Address Translation (ADDR_TRANSLATION_ON=0)

When address translation is disabled, the incoming AXI addresses are passed to the NOC interface as-is without any adjustment or translation. In this case the TLB tables are not used. The NOC capabilities are specified directly via the AxUSER input signals as described in section 2.3.2.1.1.

### 2.6.2 With Address Translation (ADDR_TRANSLATION_ON=1)

When address translation is enabled, both the addresses and the characteristics of the outgoing NOC transactions are determined partially from the incoming AXI addresses and partially from programmable "TLB" tables in the NOC2AXI SS. Figure 3summarizes how the output NOC address bits are generated from the input AXI addresses.

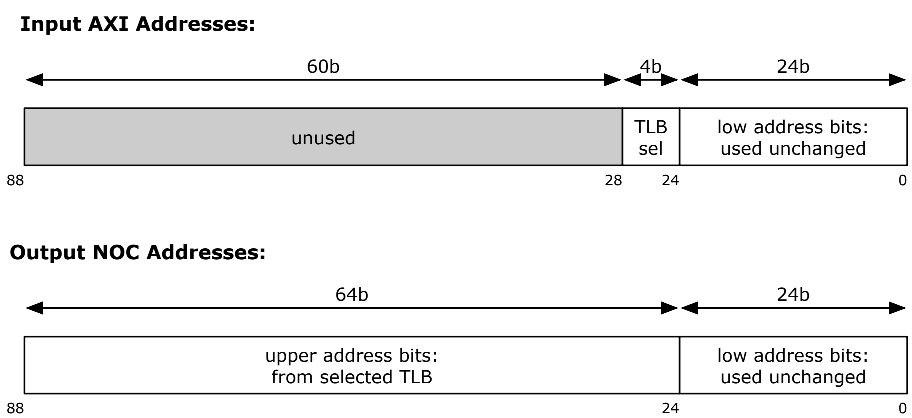


**Figure 3: AXI-to-NOC Address Translation**

#### *2.6.2.1 TLB Selection*

The NOC2AXI SS contains sixteen tables that define possible combinations of the upper NOC address values and the characteristics of the NOC transactions. Bits 27 through 24 of the input AXI address select which of the sixteen table entries should be used for the outgoing NOC transaction.

#### *2.6.2.2 Upper Address Bits*

The upper 64 bits of the output NOC address are taken from the selected TLB entry.

Note that the upper 60 bits of the input AXI address are ignored when address translation is enabled.

#### *2.6.2.3 Lower Address Bits*

The lower 24 bits of the output NOC address are taken directly from the lower 24 bits of the input AXI address. The TLB entry does not contain any bits corresponding to these lower address bits.

#### *2.6.2.4 SIdeband Signals (AxUSER)*

The sideband signals are the set of signals that define characteristics of the outgoing NOC transaction, such as whether it is a multicast transaction or a posted transaction, etc. Each TLB entry contains 38 bits of sideband signal data, which are mapped to the internal AxUSER signals as shown in Table 2. For example, sideband bit 1 specifies strict ordering, and sideband bit 2 specifies a posted transaction.

Note that the input AxUSER signals are ignored if address translation is enabled.

## 2.7 AXI CDC

Two AXI clock domain crossing modules are used to convert AXI transactions between the AXI and NOC clock domains: one for NOC-to-AXI traffic, and one for AXI-to-NOC traffic. Each of these modules comprises a set of CDC FIFOs, one for each AXI channel.

## 2.8 Register Infrastructure

Figure 4 shows the register infrastructure in the NOC2AXI SS.

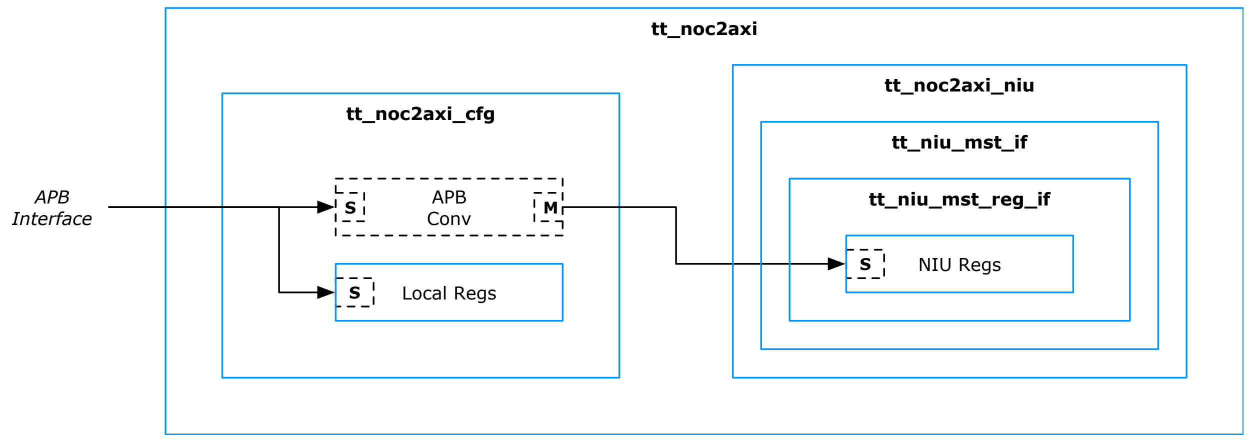


**Figure 4: Register Infrastructure**

## 2.9 Latency

**TODO**

# 3 Signal Description

This section describes the signal interface of the NOC2AXI SS.

| Name | Width | Description |
|---|---|---|
| **Constant Configuration Inputs** | | |
| i_dateline_node_* | 1 | |
| i_noc_*_size | 6 | |
| i_local_nocid | 6 | |
| i_local_nodeid_* | 6 | |
| i_noc_endpoint_id | | |
| **Global Interface** | | |
| i_nocclk | 1 | NOC clock |
| i_axiclk | 1 | AXI clock |
| i_reset_n | 1 | Main reset |
| i_MEM_*_Settings | | |
| o_ecc_error | 3 | |
| o_tile_clk_off | 1 | |
| o_intp_id_transaction_count_rtz | 1 | |
| **APB Interface** | | |
| i_reg_psel | 1 | |
| i_reg_penable | 1 | |
| i_reg_paddr | 32 | |
| i_reg_pwrite | 1 | |
| i_reg_pwdata | 32 | |
| o_reg_pready | 1 | |
| o_reg_prdata | 32 | |
| o_reg_pslverr | 1 | |
| **NOC Interface** | | |
| o_chan_flit | 515 | |
| o_chan_vld | 1 | |
| o_chan_vc | 4 | |
| i_chan_credit | 16 | |
| i_chan_backoff | 16 | |
| i_chan_flit | 515 | |
| i_chan_vld | 1 | |
| i_chan_vc | 4 | |
| o_chan_credit | 16 | |
| o_chan_backoff | 16 | |
| i_security_fence_en | 1 | |
| i_routing_dim_order_xy | 1 | |
| **NOC to AXI Interface** | | |
| o_noc2axi_ar* | | |
| i_noc2axi_arready | 1 | |
| i_noc2axi_r* | | |
| o_noc2axi_rready | 1 | |
| o_noc2axi_aw* | | |
| i_noc2axi_awready | 1 | |
| o_noc2axi_w* | | |

| | | |
|---|---|---|
| i_noc2axi_wready | 1 | |
| i_noc2axi_b* | | |
| o_noc2axi_bready | 1 | |
| **AXI to NOC Interface** | | |
| i_noc2axi_ar* | | |
| o_noc2axi_arready | 1 | |
| o_noc2axi_r* | | |
| i_noc2axi_rready | 1 | |
| i_noc2axi_aw* | | |
| o_noc2axi_awready | 1 | |
| i_noc2axi_w* | | |
| o_noc2axi_wready | 1 | |
| o_noc2axi_b* | | |
| i_noc2axi_bready | 1 | |
| **DFT Interface** | | |
| i_test_clk_en | 1 | |
| i_test_mode | 1 | |
| i_scan_rst_n | 1 | |

Table 3: Signal Interface

# 4 Software Interface

## 4.1 Address Map

Table 4 shows the full address map for the NOC2AXI SS.

| Region | Address Range | Size |
| --- | --- | --- |
| NIU registers | 0x0000_0000 - 0x0000_0FFF | 4 KB |
| local registers | 0x0000_1000 - 0x0000_10FF | 256 B |
| [reserved] | 0x0000_1100 - 0xFFFF_FFFF | |

**Table 4: NOC2AXI SS Register Map**

## 4.2 Programming Sequences

**TODO**

# 5 Implementation

## 5.1 Top-Level Parameters

Table 5 shows the top-level parameters of the NOC2AXI SS.

| Parameter Name | Default Value | Description |
|---|---|---|
| ADDR_TRANSLATION_ON | 0 | Enables translation / TLB lookup for AXI-to-NOC addresses |
| AXI_DATA_WIDTH | 512 | Bit width of AXI data bus |

**Table 5: Top-level Parameters**

## 5.2 Clocks

Table 6 lists the main functional clocks in the NOC2AXI SS. Gated versions are excluded from the list for the sake of brevity.

| Signal Name | Max Func Freq | Logic |
|---|---|---|
| i_nocclk | 1.4 GHz | NOC router, NIU |
| i_axiclk | 1.4 GHz | AXI interface |

**Table 6: Clocks**

Figure 5 shows the distribution of the clocks, including the clock gating cells.

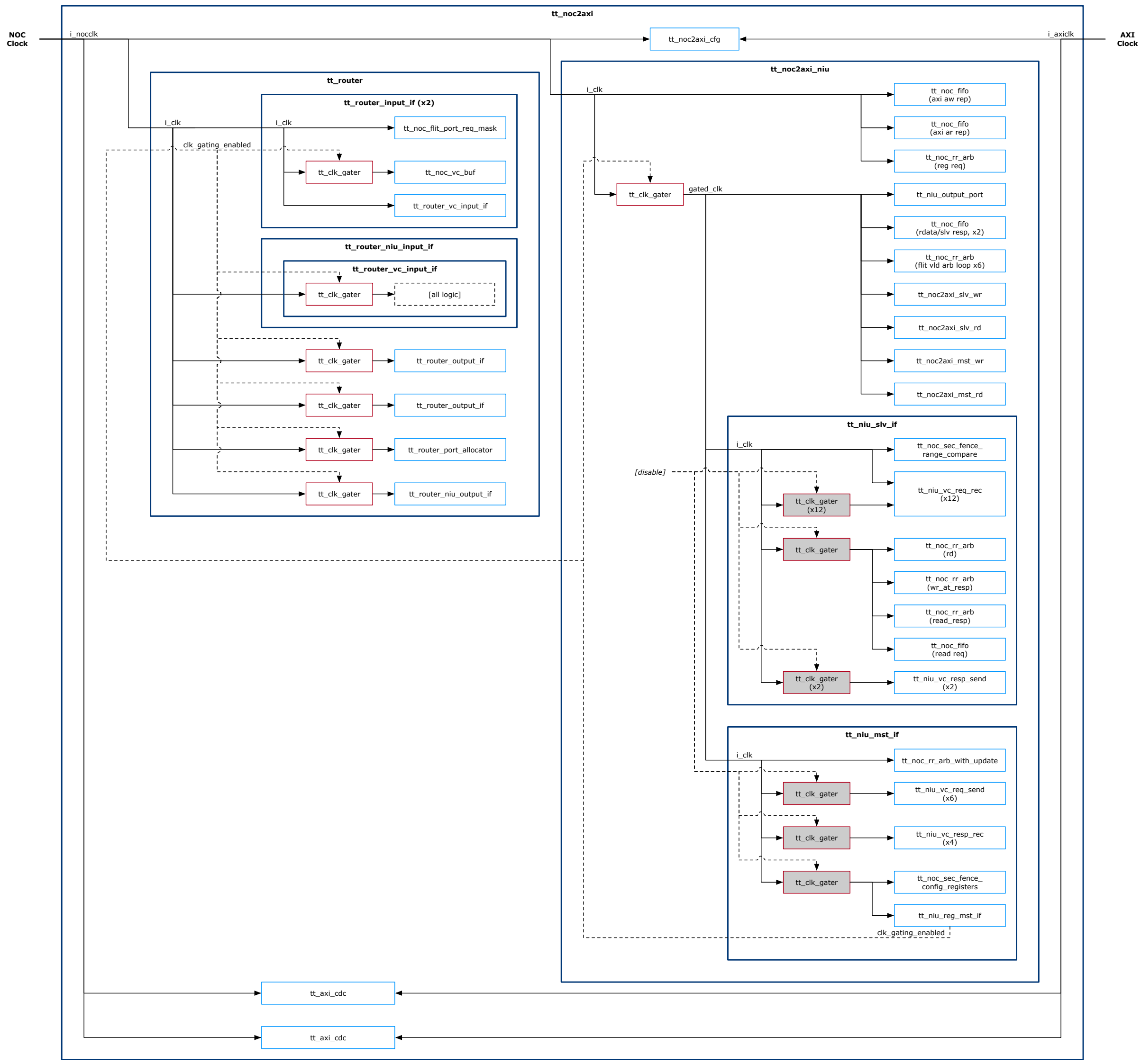

Figure 5: Clocking Structure

### 5.2.1 Clock Gating

Clock gating cells are present in the NOC2AXI SS to help reduce the dynamic power consumption. All clock gating cells implement dynamic clock gating, whereby the hardware itself detects when valid activity is no longer occurring and allows the clocks to be temporarily turned off.

None of the clock gating cells are statically-controlled: the clocks cannot be explicitly disabled by software. **TODO: make some of these statically-controlled!**

## 5.3 Resets

Table 7 lists the hardware reset signals that are inputs to the NOC2AXI SS.

| Signal Name | Affected Logic |
|---|---|
| i_reset_n | all logic in NOC2AXI SS |

Table 7: Input Resets

Figure 6 shows the structure of the reset distribution.

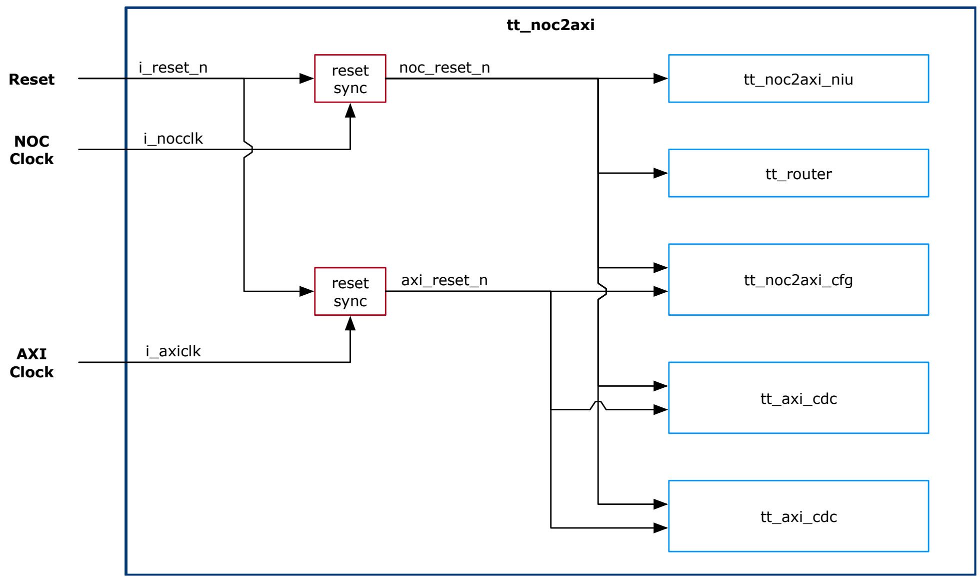


Figure 6: Reset Structure

## 5.4 Speed and Size

### 5.4.1 Area

Table 8 shows an estimate of the post-synthesis pre-PNR area of the NOC2AXI SS and its major components, using the Samsung SF4 process.

| Component(s) | Pre-PNR Area [$mm^2$] |
|---|---|
| Router | 0.139 |
| NIU | 0.159 |
| N2A / A2N CDC | 0.018 |
| **Total** | **0.317** |

Table 8: Area Estimate

## 5.5 Memories

There are memory macros located in the sub-blocks of the NOC2AXI SS. Table 9 shows the full set of memories. The nominal required size of each memory is shown, in addition to the size and breakdown of the RAM macros that are actually implemented. All bit widths include any ECC or parity bits.

| Function | Type | # of Instances | Nominal Instance Size | Physical Impl Size | Total # of Bits |
|---|---|---|---|---|---|
| NOC Router VC Buffer | two-port | 2 | 64 words x 2048 bits | 64 words x 136 bits x 16 banks | 278528 |
| AXI to NOC Write Buffer | two-port | 1 | 64 words x 2048 bits | 64 words x 136 bits x 16 banks | 139264 |
| AXI to NOC Read Buffer | two-port | 1 | 256 words x 2048 bits | 256 words x 136 bits x 16 banks | 557056 |
| NOC to AXI Read Buffer | two-port | 1 | 64 words x 515 bits | 64 words x 137 bits x 4 banks | 35072 |

**Table 9: Memories**